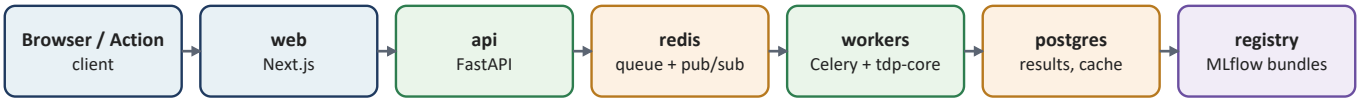
AI TECHNICAL DEBT PRIORITIZER | SUMMARY 2 OF 3

Development and DevOps

Services, data, interfaces, pipelines, deployment and operations (short version of design document 2)

1. Architecture

All analysis and ML logic lives in one Python package, **tdp-core**, used by the API workers, the CLI, the GitHub Action and the training pipeline, so training and serving compute identical features. The core never imports web frameworks.



Request path. Workers also use a repo-mirror volume, the GitHub API and the LLM API.

2. Stack

Layer	Choice	Layer	Choice
Backend, ML	Python 3.12, uv workspace	Jobs	Celery + Redis
Parsing, git	Tree-sitter, git CLI, GitPython / PyDriller	Database	PostgreSQL 16, SQLAlchemy 2, Alembic
Graphs, data	NetworkX, SciPy sparse, Pandas, Parquet	Frontend	Next.js, TypeScript, Tailwind, shadcn/ui
ML	XGBoost, scikit-learn, SHAP, Optuna	Charts	Recharts + Cytoscape.js (graph view)
Tracking	MLflow tracking + registry	LLM	Anthropic API (Claude Sonnet / Haiku)
API	FastAPI, Pydantic v2	Ops	GitHub Actions, GHCR, Docker Compose, Prometheus, Sentry

3. Analysis engine and jobs

- Stages: fetch, mine, parse, graphs, features, score, explain, plan, persist, run as a **Celery chain** with retries, late acknowledgement and time limits; separate *analysis* and *llm* queues.
- **Idempotent:** an analysis is keyed by (repository, commit SHA, model version); progress streams to the browser by **Server-Sent Events**.
- **Fast:** one streamed *git log --numstat* pass, bare mirrors fetched incrementally, static metrics cached by (blob SHA, extractor version), process pool for parsing. Targets: 5 minutes cold for 5,000 commits, 30 s incremental.
- **Limits:** skip files over 1 MB, minified, generated and vendored code; per-file and per-analysis timeouts.

4. API, data and frontend

- **API v1:** POST /analyses (202), GET status, /events (SSE), /files (ranked, cursor pagination), /files/{id}, /graph, /treemap, /plan?budget_hours, POST /whatif, /advice, /feedback, /trend, GitHub webhooks (HMAC), health and metrics. Pydantic models generate OpenAPI, which generates the TypeScript client.
- **Postgres:** repositories, analyses, file_results (indexed priority, risk, debt types; JSONB features, SHAP and reasons), plans, advice, feedback, static_cache, model_versions. Bulk COPY writes; Alembic migrations.
- **Dashboard:** overview, ranked file table (virtualised), file detail with reasons, what-if and advice, treemap, dependency graph, budgeted plan page, trend page. Server components plus TanStack Query; colour-blind-safe bands with text labels.

5. MLOps

- **tdp-ml** commands: manifest (repos pinned to SHAs), mine, features, labels, train (Optuna + MLflow), evaluate, register, promote.
- **Model bundle** of JSON/YAML only (XGBoost JSON, calibrator arrays, feature schema, reason templates, cost model, metadata): no pickle, schema checked on load.
- **Promotion gates:** no NDCG regression, beats the hotspot baseline ($p < 0.05$), calibration error at most 0.05, leakage tests pass. Train/serve parity test; weekly PSI drift check; model card per version.

6. CLI and GitHub integration

- **CLI:** tdp analyze, tdp plan --budget 40, tdp diff --base main, tdp report --format sarif.
- **GitHub Action:** runs inside the user's CI (code never leaves it), posts one pull-request comment (updated in place) when Critical or High files are touched, uploads **SARIF** to code scanning.

7. Quality, delivery and deployment

- **Tests:** known-value fixtures, scripted temporary git repos (churn, SZZ), Hypothesis for the knapsack, API tests with Postgres and Redis containers, Vitest, Playwright end to end; 80% coverage gate on tdp-core.

- **CI/CD:** ruff, mypy, ESLint and type checks, tests, image build, Trivy scan and SBOM, push to GHCR, auto deploy to staging, approved deploy to production; pre-commit, Dependabot, Actions pinned to SHAs.
- **Containers:** Compose with migrate, api, worker, worker-llm, web, db, redis (and mlflow profile); non-root images; one command: *make up*.
- **Deployment:** recommended single VM with Compose and Caddy (automatic HTTPS) for the demo; optional AWS CDK stack (ECS Fargate, RDS, ElastiCache, S3, Secrets Manager) deployed on demand.

8. Security and observability

- **Untrusted input:** https-only host allowlist with private-address blocking; bare mirrors, hooks disabled, no submodules; repository code is never executed; size, time and file limits; LLM gets quoted data, no tools, schema-checked output; secrets server-side only; rate limits; read-only GitHub App tokens.
- **Observability:** structlog JSON logs keyed by analysis id, Prometheus metrics (stage durations, queue depth, cache hit ratio, LLM tokens, drift), Sentry, Flower, one Grafana board with alerts.

9. Plan (12 weeks)

Weeks	ML track	Platform track
1 to 3	repo manifest, miner, static, graph and history features	monorepo, CI, Compose, FastAPI and Celery skeleton
4 to 6	SZZ labels, dataset, baselines, evaluation harness, ranker	analysis endpoints, SSE, dashboard shell and file pages: **end-to-end slice**
7 to 9	calibration, SHAP, ablation, cost model, knapsack, SATD, bundles	treemap, graph, plan and what-if pages, caching, metrics
10 to 12	LLM advisor, sensitivity analysis, final experiments	Action, SARIF, security hardening, deployment, report and demo